



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

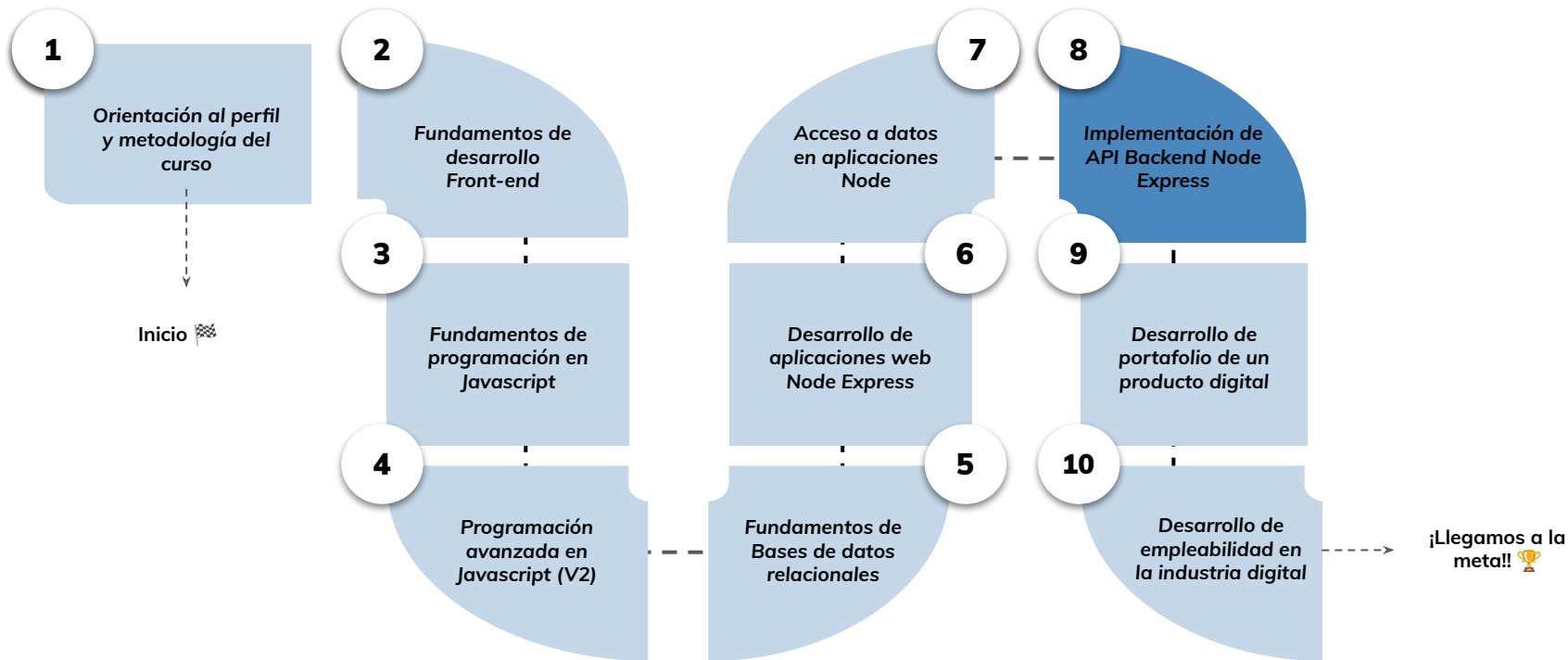


› Clase Apis Restful

AE1.1: Caracterizar los elementos fundamentales de una arquitectura rest distinguiendo buenas prácticas para el diseño de una api rest para la interoperación de sistemas.

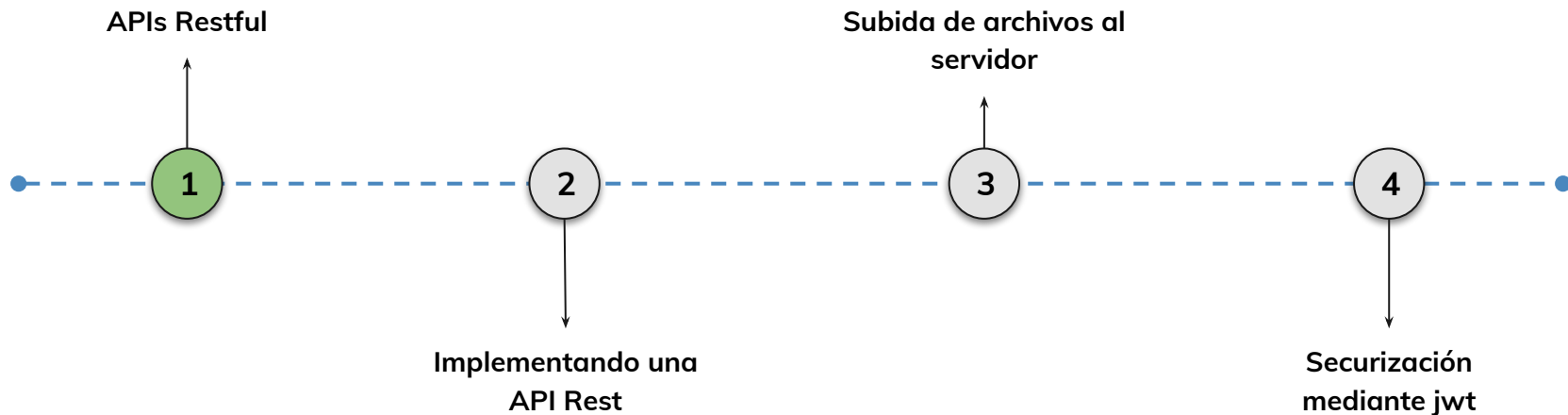
Hoja de ruta

¿Cuáles **módulos** conforman el programa?



Roadmap de la lección

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Aprendizaje N° 1: Comprender qué es una API RESTful y para qué se utiliza.
- Aprendizaje N° 2: Identificar las características y reglas fundamentales de una arquitectura REST.
- Aprendizaje N° 3: Aplicar buenas prácticas en la construcción de rutas, recursos y manejo de respuestas HTTP.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprender qué es una relación uno a uno en el contexto de bases de datos relacionales.
- Identificar cuándo es conveniente usar este tipo de relación en un modelo de datos.
- Implementar una relación uno a uno en Sequelize utilizando `hasOne` y `belongsTo`.
- Configurar claves foráneas y validar el comportamiento de la relación en operaciones CRUD.

Introducción a RESTful



Introducción a RESTful



-  ¿Qué significa REST y para qué sirve?
- **REST** (Representational State Transfer) es un estilo arquitectónico para diseñar servicios web. Fue propuesto por **Roy Fielding** en el año 2000 y se basa en los principios del protocolo HTTP.
-  ¿Para qué sirve?
- REST permite que diferentes aplicaciones, incluso desarrolladas en lenguajes distintos, se comuniquen entre sí a través de Internet usando **peticiones HTTP estándar** (GET, POST, PUT, DELETE, etc.).
- Se utiliza principalmente para crear **APIs RESTful**, que exponen datos y operaciones como recursos accesibles desde URLs.



Introducción a RESTful



Características principales de REST

1. **Interfaz uniforme**
Todo recurso debe tener una URL única y clara. Se usan los verbos HTTP para interactuar con esos recursos.
2. **Sin estado (stateless)**
Cada petición debe contener toda la información necesaria. El servidor no guarda el estado del cliente entre peticiones.
3. **Cliente-servidor**
Separación de responsabilidades: el cliente maneja la interfaz de usuario y el servidor gestiona los datos.



Introducción a RESTful



4. **Caché**

Las respuestas pueden marcarse como cacheables para mejorar el rendimiento y reducir la carga del servidor.

5. **Sistema en capas (Layered System)**

La arquitectura puede tener diferentes capas como balanceadores, firewalls o cachés intermedios.

6. **Código bajo demanda (opcional)**

El servidor puede enviar código ejecutable (por ejemplo, JavaScript) al cliente, si se requiere.



Introducción a RESTful



Ventajas y desventajas del enfoque RESTful

Ventajas:

- **Simplicidad:** Usa HTTP estándar, fácil de aprender y de implementar.
- **Flexibilidad:** Compatible con cualquier cliente que entienda HTTP (navegadores, apps móviles, etc.).
- **Escalabilidad:** La arquitectura sin estado permite balanceo de carga y replicación.
- **Interoperabilidad:** Permite que sistemas heterogéneos trabajen juntos fácilmente.
- **Mantenimiento claro:** Separación entre cliente y servidor.



Introducción a RESTful

✗ Desventajas:

- **Sin estado:** A veces obliga a repetir datos en cada solicitud (por ejemplo, tokens de autenticación).
- **Limitado para operaciones complejas:** No maneja fácilmente flujos de trabajo de varios pasos sin acoplar lógica adicional.
- **Documentación esencial:** Si no está bien documentada, una API REST puede ser difícil de usar.
- **No apta para tiempo real:** REST está basado en solicitudes y respuestas; para tiempo real es preferible WebSockets.



Introducción a RESTful



Acción	Verbo HTTP	URL
Obtener todos los usuarios	GET	/usuarios
Crear un nuevo usuario	POST	/usuarios
Obtener un usuario	GET	/usuarios/1
Actualizar un usuario	PUT	/usuarios/1
Eliminar un usuario	DELETE	/usuarios/1



Introducción a RESTful



✗ Desventajas:

- **Sin estado:** A veces obliga a repetir datos en cada solicitud (por ejemplo, tokens de autenticación).
- **Limitado para operaciones complejas:** No maneja fácilmente flujos de trabajo de varios pasos sin acoplar lógica adicional.
- **Documentación esencial:** Si no está bien documentada, una API REST puede ser difícil de usar.
- **No apta para tiempo real:** REST está basado en solicitudes y respuestas; para tiempo real es preferible WebSockets.

Explorando una API RESTful pública

¿En qué consistirá la Demo?

En esta demo teórica analizaremos una API REST pública para identificar las reglas y características que definen a una API RESTful bien estructurada.



Consigna:

Explorá un endpoint como

<https://jsonplaceholder.typicode.com/users> y respondé:

- ¿Qué verbo HTTP se utiliza?
- ¿Qué estructura tiene la URL?
- ¿Qué tipo de respuesta devuelve la API?
- ¿Qué reglas RESTful podés identificar?



Paso a paso:

1. Ingresá a <https://jsonplaceholder.typicode.com>.
2. Seleccioná un recurso como [/users](#) o [/posts](#).
3. Observá el tipo de respuesta que devuelve (JSON).
4. Analizá el verbo HTTP y el código de estado (podés usar Postman o tu navegador).
5. Reflexioná sobre qué principios RESTful se aplican en este endpoint.

Tiempo: 20 minutos



Momento:

Time-out!



5 -10 min.



Reglas fundamentales de una arquitectura REST



Reglas fundamentales de una arquitectura REST

✓ 1. Interfaz uniforme con mensajes descriptivos

Esta regla establece que toda comunicación con la API debe seguir una **estructura clara y predecible**, lo que facilita su uso por cualquier cliente (navegador, aplicación móvil, otro servidor).

Principios clave:

- Cada recurso debe tener una **URL única** y representar un **sustantivo** (ej: [/usuarios](#)).
- Se utilizan los **verbos HTTP** para definir la acción (GET, POST, PUT, DELETE).
- Las respuestas deben ser **autodescriptivas**, con códigos de estado HTTP y mensajes claros.



Reglas fundamentales de una arquitectura REST

Ejemplo:

```
GET /usuarios/15  
→ 200 OK  
{  
  "id": 15,  
  "nombre": "Carla"  
}
```



Reglas fundamentales de una arquitectura REST

✓ 2. Peticiones sin estado (Stateless)

Cada petición HTTP de un cliente debe contener **toda la información necesaria** para que el servidor la entienda. El servidor **no guarda el estado** entre peticiones.

Implicancia:

- No hay sesiones ni contexto almacenado en el servidor.
- Requiere enviar tokens u otros datos en cada solicitud si se necesita autenticación.

Ventajas:

- Escalabilidad (fácil balancear carga entre servidores).
- Menor complejidad en el backend.



Reglas fundamentales de una arquitectura REST



✓ 3. Caché (Cacheable)

Las respuestas de la API pueden indicar si son **cacheables** mediante cabeceras HTTP (**Cache-Control**, **ETag**), para que el cliente o un proxy pueda reutilizarlas.

Beneficios:

- Menor carga en el servidor.
- Mejores tiempos de respuesta.

Ejemplo de cabecera:

Cache-Control: max-age=3600

Significa que la respuesta puede guardarse por una hora.



Reglas fundamentales de una arquitectura REST



4. Separación clara Cliente-Servidor

El cliente y el servidor deben estar claramente separados:

- El **cliente** solo se encarga de la interfaz de usuario.
- El **servidor** solo maneja la lógica, los datos y las respuestas.

Ventajas:

- Independencia tecnológica (frontend en React, backend en Node.js, por ejemplo).
- Desarrollo paralelo entre equipos.
- Mantenimiento más simple.



Reglas fundamentales de una arquitectura REST

✓ 5. Sistema de capas (Layered System)

Una arquitectura REST puede estar compuesta por **capas jerárquicas** como:

- Balanceadores de carga
- Caches intermedios
- Gateways de seguridad
- Servidores de aplicación

El cliente **no necesita saber qué capa está manejando su solicitud.**

Ventajas:

- Mejora la escalabilidad y el control de seguridad.
- Permite ocultar la complejidad del sistema interno.



Ejercicio N° 01

Reglas fundamentales de una arquitectura REST

Reglas fundamentales de una arquitectura REST

Contexto: 🙌

Las arquitecturas RESTful siguen un conjunto de reglas que aseguran escalabilidad, simplicidad y eficiencia. En esta demo, vamos a identificar en un caso real cómo se implementan esas reglas.

Consigna: 📝

Accedé a <https://jsonplaceholder.typicode.com/posts> y respondé:

- ¿La URL es clara y descriptiva?
- ¿Qué indica el método HTTP?
- ¿La respuesta incluye encabezados relacionados con el cache?
- ¿Hay estado mantenido entre peticiones?
- ¿Quién controla el flujo: el cliente o el servidor?
- ¿Podrías imaginar que esta API forma parte de un sistema por capas?

Tiempo 🕒: 35 minutos

Reglas fundamentales de una arquitectura REST

Paso a paso:

1. Entrá a <https://jsonplaceholder.typicode.com/posts>.
2. Observá la estructura del recurso: `/posts` es el nombre del recurso.
3. Analizá la respuesta: formato JSON, encabezados, códigos de estado.
4. Abrí herramientas de desarrollo (F12 → pestaña Network) y revisá los headers.
5. Verificá si hay headers como `Cache-Control`, `ETag`, etc.
6. Realizá dos peticiones seguidas y comprobá que no hay estado mantenido.
7. Discutí en grupo cómo se aplican cada una de las **5 reglas REST**.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Logro 1:** Comprendiste el concepto de REST y su aplicación en el diseño de APIs.
- ✓ **Logro 2:** Identificaste las reglas clave que debe cumplir una arquitectura RESTful.
- ✓ **Logro 3:** Reconociste la utilidad de los verbos HTTP y la definición correcta de recursos.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

